

LAPORAN PRAKTIKUM

STRUKTUR DATA

Doubly Linked List

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHARAN AZARIA ANVAYA



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 14 MEI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Doubly Linked List**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Doubly Linked List.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 14 Mei 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Kode Program	2
2.2.1 NodeDLL	2
2.2.2 InsertDLL.....	2
2.2.3 PenelusuranDLL	4
2.2.4 HapusDLL	5
BAB III KESIMPULAN	8
3.1 Kesimpulan.....	8
DAFTAR PUSTAKA.....	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data adalah cara untuk menyimpan dan mengatur data sehingga Anda dapat menggunakannya secara efisien. Linked list adalah struktur data linear yang terdiri dari kumpulan elemen yang disebut node, di mana setiap node menyimpan dua bagian utama yaitu data dan referensi (pointer) ke node berikutnya. Doubly linked list adalah jenis daftar berantai di mana node berisi informasi dan dua pointer, yaitu pointer kiri dan pointer kanan.

1.2 Tujuan Praktikum

- Memahami konsep *Abstract Data Type (ADT) Doubly Linked List* dalam pemrograman.
- Mampu merancang dan mengimplementasikan *Doubly Linked List* menggunakan bahasa Java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan *Doubly Linked List* dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II PEMBAHASAN

2.1 Dasar Teori

Doubly Linked List adalah struktur data yang memiliki referensi ke node sebelumnya dan node berikutnya dalam daftar. Struktur ini memberikan kemudahan dalam menelusuri, menyisipkan, dan menghapus node dalam daftar ke kedua arah. Dalam daftar berantai ganda, setiap node berisi tiga anggota data yaitu: data, next dan prev. Linked list mendukung operasi penyisipan dan penghapusan yang efisien. Beberapa contoh operasi dasar di linked list yaitu traversal, insertion, deletion, searching, updating, dan reversal.

2.2 Kode Program

2.2.1 NodeDLL

```
package pekan6_2511533007;

public class NodeDLL_2511533007 {
    //mendefinisikan kelas Node
    int data_3007; // data
    NodeDLL_2511533007 next_3007; //pointer ke next node
    NodeDLL_2511533007 prev_3007; //pointer ke previous node

    // konstruktor
    public NodeDLL_2511533007 (int data_3007) {
        this.data_3007 = data_3007;
        this.next_3007 = null;
        this.prev_3007 = null;
    }
}
```

Kode Program 2.1

Program ini berfungsi ADT dari sebuah Node. Kode ini mendefinisikan sebuah Node untuk DLL yang terdiri dari data, next, dan prev. Pointer ini memungkinkan navigasi dua arah—maju ke elemen berikutnya dan mundur ke elemen sebelumnya. Saat objek dibuat, data akan diisi sementara kedua pointernya diatur ke null sebagai tanda bahwa node tersebut belum terhubung ke node lain..

2.2.2 InsertDLL

```
package pekan6_2511533007;

public class InsertDLL_2511533007 {
    //menambahkan node di awal DLL
    static NodeDLL_2511533007 insertBegin_3007(NodeDLL_2511533007 head_3007, int data_3007) {
        // buat node baru
        NodeDLL_2511533007 new_node_3007 = new NodeDLL_2511533007(data_3007);
        // jadikan pointer nextnya head
        new_node_3007.next_3007 = head_3007;
        // jadikan pointer prev head ke new_node
        if (head_3007 != null) {
            head_3007.prev_3007 = new_node_3007;
        }
        return new_node_3007;
    }
    // fungsi menambahkan node di akhir
}
```

```

        public static NodeDLL_2511533007 insertEnd_3007(NodeDLL_2511533007 head_3007, int
newData_3007) {
            // buat node baru
            NodeDLL_2511533007 newNode_3007 = new NodeDLL_2511533007 (newData_3007);
            // jika dll null jadikan head
            if (head_3007 == null) {
                head_3007 = newNode_3007;
            }
            else {
                NodeDLL_2511533007 curr_3007 = head_3007;
                while (curr_3007.next_3007 != null) {
                    curr_3007 = curr_3007.next_3007;
                }
                curr_3007.next_3007 = newNode_3007;
                newNode_3007.prev_3007 = curr_3007;
            }
            return head_3007;
        }
        // fungsi menambahkan node di posisi tertentu
        public static NodeDLL_2511533007 insertAtPosition_3007(NodeDLL_2511533007 head_3007, int
pos_3007, int new_data_3007) {
            // buat node baru
            NodeDLL_2511533007 new_node_3007 = new NodeDLL_2511533007(new_data_3007);
            if (pos_3007 == 1) {
                new_node_3007.next_3007 = head_3007;
                if (head_3007 != null) {
                    head_3007.prev_3007 = new_node_3007;
                }
                head_3007 = new_node_3007;
                return head_3007;
            }
            NodeDLL_2511533007 curr_3007 = head_3007;
            for (int i = 1; i < pos_3007 - 1 && curr_3007 != null; ++i) {
                curr_3007 = curr_3007.next_3007;
            }
            if (curr_3007 == null) {
                System.out.println("posisi tidak ada");
                return head_3007;
            }
            new_node_3007.prev_3007 = curr_3007;
            new_node_3007.next_3007 = curr_3007.next_3007;
            curr_3007.next_3007 = new_node_3007;
            if (new_node_3007.next_3007 != null) {
                new_node_3007.next_3007.prev_3007 = new_node_3007;
            }
            return head_3007;
        }
        public static void printList_3007(NodeDLL_2511533007 head_3007) {
            NodeDLL_2511533007 curr_3007 = head_3007;
            while (curr_3007 != null) {
                System.out.print(curr_3007.data_3007 + " <-> ");
                curr_3007 = curr_3007.next_3007;
            }
            System.out.println();
        }
        public static void main(String[] args) {
            // membuat dll 2 <-> 3 <-> 5
            NodeDLL_2511533007 head_3007 = new NodeDLL_2511533007(2);
            head_3007.next_3007 = new NodeDLL_2511533007(3);
            head_3007.next_3007.prev_3007 = head_3007;
            head_3007.next_3007.next_3007 = new NodeDLL_2511533007(5);
            head_3007.next_3007.next_3007.prev_3007 = head_3007.next_3007;
            // cetak DLL awal
            System.out.print("DLL Awal: ");
            printList_3007(head_3007);
            // tambah 1 di awal
            head_3007 = insertBegin_3007(head_3007, 1);
            System.out.print("simpul 1 ditambah di awal: ");
            printList_3007(head_3007);
            // tambah 6 di akhir
            System.out.print("simpul 6 ditambah di akhir: ");
            int data_3007 = 6;
            head_3007 = insertEnd_3007(head_3007, data_3007);
            printList_3007(head_3007);
            // menambahkan node 4 di posisi 4

```

```

        System.out.print("tambah node 4 di posisi 4: ");
        int data2_3007 = 4;
        int pos_3007 = 4;
        head_3007 = insertAtPosition_3007(head_3007, pos_3007, data2_3007);
        printList_3007(head_3007);
    }
}

```

Kode Program 2.2

Kode ini mengimplementasikan tiga method insert pada *Doubly Linked List*, di awal (insertBegin), di akhir (insertEnd), dan pada posisi yang ditentukan (insertAtPosition). Setiap fungsi bertugas mengatur ulang pointer next dan prev agar hubungan dua arah antar node tetap terjaga, termasuk menangani kondisi khusus seperti list yang masih kosong atau posisi yang tidak ditemukan. printList berfungsi untuk menampilkan list yang dibuat

```

DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

gambar 1

2.2.3 PenelusuranDLL

```

package pekan6_2511533007;

public class PenelusuranDLL_2511533007 {
    // fungsi penelusuran maju
    static void forwardTraversal_3007(NodeDLL_2511533007 head_3007) {
        // memulai penelusuran dari head
        NodeDLL_2511533007 curr_3007 = head_3007;
        // lanjutkan sampai akhir
        while (curr_3007 != null) {
            //print data
            System.out.print(curr_3007.data_3007 + " <-> ");
            //pindah ke node berikutnya
            curr_3007 = curr_3007.next_3007;
        }
        //print spasi
        System.out.println();
    }
    // fungsi penelusuran mundur
    static void backwardTraversal_3007(NodeDLL_2511533007 tail_3007) {
        // mulai dari akhir
        NodeDLL_2511533007 curr_3007 = tail_3007;
        //lanjut sampai head
        while (curr_3007 != null) {
            //cetak data
            System.out.print(curr_3007.data_3007 + " <-> ");
            // pindah ke node sebelumnya
            curr_3007 = curr_3007.prev_3007;
        }
        //cetak spasi
        System.out.println();
    }
    public static void main(String[] args) {
        //cetak dll
        NodeDLL_2511533007 head_3007 = new NodeDLL_2511533007(1);
        NodeDLL_2511533007 second_3007 = new NodeDLL_2511533007(2);
        NodeDLL_2511533007 third_3007 = new NodeDLL_2511533007(3);

        head_3007.next_3007 = second_3007;
        second_3007.prev_3007 = head_3007;
    }
}

```

```

        second_3007.next_3007 = third_3007;
        third_3007.prev_3007 = second_3007;

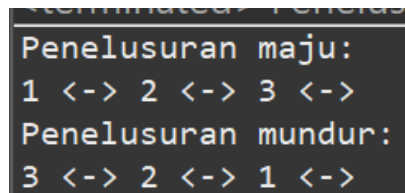
        System.out.println("Penelusuran maju: ");
        forwardTraversal_3007(head_3007);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_3007(third_3007);
    }
}

```

Kode Program 2.3

Kode ini mengimplementasikan penelusuran dua arah pada *Doubly Linked List* menggunakan fungsi `forwardTraversal` (penelusuran maju) dan `backwardTraversal` (penelusuran mundur). Melalui contoh pada method `main`, program membuktikan bahwa data dapat diakses secara berurutan dari kedua ujung daftar berkat adanya koneksi antar simpul hingga mencapai nilai `null`.



```

Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->

```

gambar 2

2.2.4 HapusDLL

```

package pekan6_2511533007;

public class HapusDLL_2511533007 {
    // fungsi menghapus node awal
    public static NodeDLL_2511533007 delHead_3007(NodeDLL_2511533007 head_3007) {
        if (head_3007 == null) {
            return null;
        }
        NodeDLL_2511533007 temp_3007 = head_3007;
        head_3007 = head_3007.next_3007;
        if (head_3007 != null) {
            head_3007.prev_3007 = null;
        }
        return head_3007;
    }
    // fungsi menghapus di akhir
    public static NodeDLL_2511533007 delLast_3007(NodeDLL_2511533007 head_3007) {
        if (head_3007 == null) {
            return null;
        }
        if (head_3007.next_3007 == null) {
            return null;
        }
        NodeDLL_2511533007 curr_3007 = head_3007;
        while (curr_3007.next_3007 != null) {
            curr_3007 = curr_3007.next_3007;
        }
        // update pointer previous node
        if (curr_3007.prev_3007 != null) {
            curr_3007.prev_3007.next_3007 = null;
        }
        return head_3007;
    }
    // fungsi menghapus node posisi tertentu
    public static NodeDLL_2511533007 delPos_3007(NodeDLL_2511533007 head_3007, int pos_3007) {

```



```

// jika DLL kosong
if (head_3007 == null) {
    return head_3007;
}
NodeDLL_2511533007 curr_3007 = head_3007;
// telusuri sampai ke node yang akan dihapus
for (int i = 1; curr_3007 != null && i < pos_3007; ++i) {
    curr_3007 = curr_3007.next_3007;
}
//jika posisi tidak ditemukan
if (curr_3007 == null) {
    return head_3007;
}
// update pointer
if (curr_3007.prev_3007 != null) {
    curr_3007.prev_3007.next_3007 = curr_3007.next_3007;
}
if (curr_3007.next_3007 != null) {
    curr_3007.next_3007.prev_3007 = curr_3007.prev_3007;
}
//jika yang dihapus head
if (head_3007 == curr_3007) {
    head_3007 = curr_3007.next_3007;
}
return head_3007;
}
// fungsi mencetak DLL
public static void printList_3007 (NodeDLL_2511533007 head_3007) {
    NodeDLL_2511533007 curr_3007 = head_3007;
    while (curr_3007 != null) {
        System.out.print(curr_3007.data_3007 + " <-> ");
        curr_3007 = curr_3007.next_3007;
    }
    System.out.println();
}

public static void main(String[] args) {
    // buat sebuah DLL
    NodeDLL_2511533007 head_3007 = new NodeDLL_2511533007(1);
    head_3007.next_3007 = new NodeDLL_2511533007(2);
    head_3007.next_3007.prev_3007 = head_3007;
    head_3007.next_3007.next_3007 = new NodeDLL_2511533007(3);
    head_3007.next_3007.next_3007.prev_3007 = head_3007.next_3007;
    head_3007.next_3007.next_3007.next_3007 = new NodeDLL_2511533007(4);
    head_3007.next_3007.next_3007.next_3007.prev_3007 = head_3007.next_3007.next_3007;
    head_3007.next_3007.next_3007.next_3007.next_3007 = new NodeDLL_2511533007(5);
    head_3007.next_3007.next_3007.next_3007.next_3007.prev_3007 =
head_3007.next_3007.next_3007.next_3007;

    System.out.print("DLL Awal: ");
    printList_3007(head_3007);

    System.out.print("Setelah head dihapus: ");
    head_3007 = delHead_3007(head_3007);
    printList_3007(head_3007);

    System.out.print("Setelah node terakhir dihapus: ");
    head_3007 = delLast_3007(head_3007);
    printList_3007(head_3007);

    System.out.print("menghapus node ke 2: ");
    head_3007 = delPos_3007(head_3007, 2);
    printList_3007(head_3007);
}
}

```

Kode Program 2.4

Kode ini memiliki 3 function untuk menghapus elemen dalam *Doubly Linked List* yaitu: penghapusan di awal (*delHead*), di akhir (*delLast*), dan pada posisi spesifik (*delPos*). Logika

intinya yaitu pada pemutusan hubungan node yang di mau dengan cara mengatur ulang pointer next dan prev. Dalam method main, kode menampilkan penghapusan data posisi *head*, penghapusan data terakhir, dan penghapusan posisi ke 2 dari list.

```
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->  
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->  
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->  
menghapus node ke 2: 2 <-> 4 <->
```

gambar 3

BAB III

KESIMPULAN

3.1 Kesimpulan

DLL merupakan struktur data linear yang terdiri dari sekumpulan simpul node, di mana setiap simpul memiliki dua pointer yaitu next dan prev. Struktur ini memberikan kelebihan dibandingkan SLL karena memungkinkan penelusuran data dilakukan secara dua arah, yaitu secara forward (maju) dan backward (mundur) sehingga meningkatkan kemudahan dalam manipulasi data. DLL mempermudah operasi manipulasi data, terutama saat harus mengakses atau menghapus simpul tanpa harus mengulang penelusuran dari awal head.

Berdasarkan implementasi kode pada praktikum ini, NodeDLL sebagai ADT yang mendefinisikan struktur node untuk dipanggil kelas lain yang berguna untuk penambahan, penghapusan, serta penelusuran data. Kode tersebut menunjukkan bahwa keberhasilan manipulasi DLL sangat bergantung pada penggunaan pointer yang baik agar hubungan antar node tetap bagus.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Introduction to Doubly Linked List in Java," [Daring]. Tersedia pada: <https://www.geeksforgeeks.org/dsa/introduction-to-doubly-linked-lists-in-java/>. [Diakses: 14-Mei-2026].
- [2] GeeksforGeeks, "Applications, Advantages and Disadvantages of Doubly Linked List," [Daring]. Tersedia pada: <https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-doubly-linked-list/>. [Diakses: 14-Mei-2026].

Laporan Tugas

A. Kode ADT

```
package pekan6_2511533007;

public class Lagu_2511533007 {

    // Atribut
    private String judul_3007;
    private String penyanyi_3007;

    // Pointer
    Lagu_2511533007 next_3007;
    Lagu_2511533007 prev_3007;

    // Konstruktor
    public Lagu_2511533007(String judul_3007, String penyanyi_3007) {
        this.judul_3007 = judul_3007;
        this.penyanyi_3007 = penyanyi_3007;
        this.next_3007 = null;
        this.prev_3007 = null;
    }

    // Getter
    public String getJudul_3007() {
        return judul_3007;
    }

    public String getPenyanyi_3007() {
        return penyanyi_3007;
    }

    public Lagu_2511533007 getNext_3007() {
        return next_3007;
    }

    public Lagu_2511533007 getPrev_3007() {
        return prev_3007;
    }

    // Setter
    public void setJudul_3007(String judul_3007) {
        this.judul_3007 = judul_3007;
    }

    public void setPenyanyi_3007(String penyanyi_3007) {
        this.penyanyi_3007 = penyanyi_3007;
    }

    public void setNext_3007(Lagu_2511533007 next_3007) {
        this.next_3007 = next_3007;
    }

    public void setPrev_3007(Lagu_2511533007 prev_3007) {
        this.prev_3007 = prev_3007;
    }
}
```

B. Kode Program

```
package pekan6_2511533007;

import java.util.Scanner;

public class Musik_2511533007 {

    static Lagu_2511533007 head_3007 = null;
    static Lagu_2511533007 tail_3007 = null;
    static int counterLagu_3007 = 0;
```

```

// method tambah lagu di tail / di akhir playlist
public static void tambahLagu_3007(String judul_3007, String penyanyi_3007) {
    counterLagu_3007++;
    Lagu_2511533007 newLagu_3007 = new Lagu_2511533007(judul_3007, penyanyi_3007);

    // jika playlist masih kosong, node baru langsung jadi head sekaligus tail
    if (head_3007 == null) {
        head_3007 = newLagu_3007;
        tail_3007 = newLagu_3007;
    } else {
        // sambungkan node baru ke tail saat ini
        newLagu_3007.prev_3007 = tail_3007;
        tail_3007.next_3007 = newLagu_3007;
        tail_3007 = newLagu_3007;
    }

    System.out.println("\nLagu berhasil ditambahkan");
    System.out.println("Posisi: " + counterLagu_3007);
    System.out.println("Judul: " + judul_3007);
    System.out.println("Penyanyi: " + penyanyi_3007);
}

// method hapus lagu di head / di awal
public static void hapusLaguAwal_3007() {
    if (head_3007 == null) {
        System.out.println("\nPlaylist kosong, tidak ada lagu yang bisa dihapus");
        return;
    }

    // tampilkan lagu yang akan dihapus
    System.out.println("\nLagu pertama dihapus:");
    cetakDataLagu_3007(head_3007);

    // jika hanya ada satu lagu, playlist jadi kosong
    if (head_3007 == tail_3007) {
        head_3007 = null;
        tail_3007 = null;
    } else {
        // geser head ke node berikutnya lalu putuskan pointer backnya
        head_3007 = head_3007.next_3007;
        head_3007.prev_3007 = null;
    }
    counterLagu_3007--;
}

// method untuk menampilkan semua lagu dari awal hingga akhir
public static void tampilMaju_3007() {
    if (head_3007 == null) {
        System.out.println("\nPlaylist kosong");
        return;
    }
    System.out.println("\n=== PLAYLIST ===");
    System.out.println("");

    Lagu_2511533007 curr_3007 = head_3007;
    int posisi_3007 = 1;

    // telusuri dari head hingga null
    while (curr_3007 != null) {
        System.out.println("Posisi " + posisi_3007);
        cetakDataLagu_3007(curr_3007);
        curr_3007 = curr_3007.next_3007;
        posisi_3007++;
    }
}

// menampilkan semua lagu dari akhir ke awal
public static void tampilMundur_3007() {
    if (tail_3007 == null) {
        System.out.println("\nPlaylist kosong");
        return;
    }

    System.out.println("\n=== PLAYLIST ===");
}

```

```

        System.out.println("");

        Lagu_2511533007 curr_3007 = tail_3007;
        int posisi_3007 = counterLagu_3007;

        // telusuri dari tail mundur menggunakan pointer prev hingga null
        while (curr_3007 != null) {
            System.out.println("Posisi " + posisi_3007);
            cetakDataLagu_3007(curr_3007);
            curr_3007 = curr_3007.prev_3007;
            posisi_3007--;
        }

        // mencari lagu dengan judul dan tidak case sensitive
        public static void cariLagu_3007(String judulCari_3007) {
            if (head_3007 == null) {
                System.out.println("\nPlaylist kosong, pencarian tidak dapat dilakukan");
                return;
            }

            Lagu_2511533007 curr_3007 = head_3007;
            boolean ditemukan_3007 = false;
            int posisi_3007 = 1;

            // telusuri seluruh list dan cocokkan judul dengan ignoreCase
            while (curr_3007 != null) {
                if (curr_3007.getJudul_3007().equalsIgnoreCase(judulCari_3007)) {
                    if (!ditemukan_3007)
                        System.out.println("\nHasil pencarian untuk " +
judulCari_3007 + ":");

                    System.out.println("Posisi ke " + posisi_3007 + " dalam playlist");
                    cetakDataLagu_3007(curr_3007);
                    ditemukan_3007 = true;
                }
                curr_3007 = curr_3007.next_3007;
                posisi_3007++;
            }

            if (!ditemukan_3007)
                System.out.println("\nLagu dengan judul " + judulCari_3007 + " tidak
ditemukan");
        }

        // mencetak data satu node lagu
        private static void cetakDataLagu_3007(Lagu_2511533007 l_3007) {
            System.out.println("Judul      : " + l_3007.getJudul_3007());
            System.out.println("Penyanyi : " + l_3007.getPenyanyi_3007());
            System.out.println("");
        }

        // main
        public static void main(String[] args) {
            Scanner scanner_3007 = new Scanner(System.in);
            int pilihan_3007;

            do {
                System.out.println("=== Playlist Musik  NIM: 2511533007 ===");
                System.out.println("");
                System.out.println("1. Tambah Lagu");
                System.out.println("2. Hapus Lagu Pertama");
                System.out.println("3. Lihat Playlist (Maju)");
                System.out.println("4. Lihat Playlist (Mundur)");
                System.out.println("5. Cari Lagu");
                System.out.println("6. Keluar");
                System.out.println("");
                System.out.print("Pilihan: ");
                pilihan_3007 = scanner_3007.nextInt();
                scanner_3007.nextLine();

                switch (pilihan_3007) {
                    case 1:
                        System.out.print("Judul      : ");

```

```

        String judul_3007 = scanner_3007.nextLine();
        System.out.print("Penyanyi : ");
        String penyanyi_3007 = scanner_3007.nextLine();
        tambahLagu_3007(judul_3007, penyanyi_3007);
        break;

    case 2:
        hapusLaguAwal_3007();
        break;

    case 3:
        tampilMaju_3007();
        break;

    case 4:
        tampilMundur_3007();
        break;

    case 5:
        System.out.print("Masukkan judul yang dicari: ");
        String judulCari_3007 = scanner_3007.nextLine();
        cariLagu_3007(judulCari_3007);
        break;

    case 6:
        System.out.println("\nPlaylist berakhir");
        break;

    default:
        System.out.println("\nPilihan tidak valid, masukkan angka 1-6");
    }

    System.out.println("");
} while (pilihan_3007 != 6);

scanner_3007.close();
}
}

```

C. Penjelasan Kode

Kelas pertama Lagu_2511533007 adalah ADT atau Node yang berfungsi menyimpan informasi spesifik setiap lagu seperti judul dan nama penyanyi, serta memiliki dua atribut pointer yaitu next_3007 sebagai penunjuk ke lagu berikutnya dan prev_3007 sebagai penunjuk ke lagu sebelumnya dalam playlist, yang merupakan ciri khas struktur Double Linked List.

Kelas kedua Musik_2511533007 berisi logika dan implementasi ADT.

Penambahan Lagu (tambahLagu_3007): Menerapkan fungsi *Insert at Tail*. Program akan membuat node baru, lalu langsung menyambungkannya ke tail yang sudah terlacak tanpa perlu menelusuri seluruh list. Pointer prev node baru diarahkan ke tail lama, pointer next tail lama diarahkan ke node baru, kemudian tail digeser ke node baru tersebut.

Penghapusan Lagu (hapusLaguAwal_3007): Menerapkan fungsi *Delete Head*. Lagu yang berada di posisi paling depan akan ditampilkan datanya, kemudian head digeser ke node berikutnya dan pointer prev dari head baru diset ke null untuk memutus hubungan dengan node

yang dihapus. Program juga menangani kondisi khusus saat hanya tersisa satu lagu, di mana baik head maupun tail langsung disetkan null.

Penelusuran Maju (tampilMaju_3007): Melakukan *Forward Traversal* dengan menggunakan pointer next_3007, menelusuri dari head hingga ujung list (null) untuk menampilkan seluruh lagu secara berurutan dari awal ke akhir playlist.

Penelusuran Mundur (tampilMundur_3007): Melakukan *Backward Traversal* yang merupakan keunggulan utama Double Linked List. Penelusuran dimulai dari tail dan berjalan mundur menggunakan pointer prev_3007 hingga mencapai null, sehingga seluruh lagu ditampilkan dari akhir ke awal tanpa perlu menelusuri ulang dari depan.

Pencarian (cariLagu_3007): Menggunakan metode *Linear Search* dengan menelusuri list dari head menggunakan pointer next_3007, lalu membandingkan judul yang diinput pengguna dengan atribut judul_3007 di setiap node menggunakan equalsIgnoreCase agar pencarian tidak sensitif terhadap huruf besar atau kecil.

Interaksi Pengguna: Method main menyediakan menu yang diinput menggunakan Scanner dan diproses dengan dowhile serta switch case.

D. Output Kode

```
=== Playlist Musik NIM: 2511533007 ===  
  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
  
Pilihan:
```

gambar 4 tampilan kode

```
Pilihan: 1  
Judul   : qwerty  
Penyanyi : uiop  
  
Lagu berhasil ditambahkan  
Posisi: 1  
Judul: qwerty  
Penyanyi: uiop
```

gambar 5 output jika memilih pilihan 1

```
Pilihan: 3
|
=== PLAYLIST ===

Posisi 1
Judul    : qwerty
Penyanyi : uiop

Posisi 2
Judul    : asdf
Penyanyi : ghjkl

Posisi 3
Judul    : zxcv
Penyanyi : bnm

Posisi 4
Judul    : qazwsx
Penyanyi : edcrfv
```

gambar 6 output jika memilih pilihan 3

```
Pilihan: 4
|
=== PLAYLIST ===

Posisi 4
Judul    : qazwsx
Penyanyi : edcrfv

Posisi 3
Judul    : zxcv
Penyanyi : bnm

Posisi 2
Judul    : asdf
Penyanyi : ghjkl

Posisi 1
Judul    : qwerty
Penyanyi : uiop
```

gambar 7 output jika memilih pilihan 4

```
Pilihan: 2
|
Lagu pertama dihapus:
Judul    : qwerty
Penyanyi : uiop
```

gambar 8 output jika memilih pilihan 2

```
Pilihan: 5
Masukkan judul yang dicari: asdf

Hasil pencarian untuk asdf:
Posisi ke 1 dalam playlist
Judul      : asdf
Penyanyi   : ghjkl
```

gambar 9 output jika memilih pilihan 5

```
Pilihan: 5
Masukkan judul yang dicari: plm

Lagu dengan judul plm tidak ditemukan
```

gambar 10 output jika memilih pilihan 5 dgn data salah

```
Pilihan: 7

Pilihan tidak valid, masukkan angka 1-6
```

gambar 11 output jika memilih pilihan bukan dari 1-6

```
Pilihan: 6

Playlist berakhir
```

gambar 12 output jika memilih pilihan 6

```
Pilihan: 2
|
Playlist kosong, tidak ada lagu yang bisa dihapus
```

```
Pilihan: 3

Playlist kosong
```

```
Pilihan: 4
|
Playlist kosong
```

```
Pilihan: 5
Masukkan judul yang dicari: qwe

Playlist kosong, pencarian tidak dapat dilakukan
```

gambar 13 output jika memilih pilihan 2, 3, 4, 5 list kosong